
Oliver Krauss

Inheritance & Abstract Classes

ALD | MTD.ba VZ SS26

Hagenberg 18.03.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Recap: Where We Left Off

Before we start -- lets review the code from the last session.

From your UE2 solution: How did your **Drawing** class store different shape types? What did `drawAll()` look like? What was awkward about it?

Recap: Where We Left Off

Before we start -- lets review the code from the last session.

From your UE2 solution: How did your **Drawing** class store different shape types? What did **drawAll()** look like? What was awkward about it?

Last session you built **encapsulation** and **JavaFX** in UE2:

- Fields are **private**, accessed through getters/setters with validation
- Each shape class (**Circle**, **Rectangle**, **Line**) knows how to **draw()** itself
- **@Override toString()** for readable output, **@Override equals()** for value comparison
- Composition: **Drawing** has an array of shapes

Today: We look at a problem in the **Drawing** class -- and solve it with **inheritance** and **abstract classes**.

The Current Drawing Class

In the exercise, you built a **Drawing** that manages different shape types:

```
1  public class Drawing {
2      private Object[] shapes;
3      private int count;
4
5      public Drawing(int maxShapes) {
6          this.shapes = new Object[maxShapes];
7          this.count = 0;
8      }
9
10     public void add(Object shape) {
11         if (this.count < this.shapes.length) {
12             this.shapes[this.count] = shape;
13             this.count++;
14         }
15     }
16     // ...
17 }
```

Notice: `Object[]` --- no common base type for Circle, Rectangle, and Line yet.

The `drawAll()` Problem

To draw all shapes, we must check each object's type with **`instanceof`**:

```
1  public void drawAll(GraphicsContext gc) {  
2      for (int i = 0; i < this.count; i++) {  
3          if (this.shapes[i] instanceof Circle c) {  
4              c.draw(gc);  
5          } else if (this.shapes[i] instanceof Rectangle r) {  
6              r.draw(gc);  
7          } else if (this.shapes[i] instanceof Line l) {  
8              l.draw(gc);  
9          }  
10     }  
11 }
```

This works -- but it has a serious design problem. Can you spot it?

What's Wrong with `instanceof` Chains?

- What happens when we add a **Triangle** class?
- We must **edit** `drawAll()` to add another `instanceof` check
- What about **Star**? **Ellipse**? **Polygon**? **Arrow**?
- Every new shape type forces us to change **Drawing**

Forgetfulness

You add a **Triangle** class -- but forget to update `drawAll()`. What happens? How would you even know something is missing?

What's Wrong with `instanceof` Chains?

- What happens when we add a **Triangle** class?
- We must **edit** `drawAll()` to add another `instanceof` check
- What about **Star**? **Ellipse**? **Polygon**? **Arrow**?
- Every new shape type forces us to change **Drawing**

Forgetfulness

Your add a **Triangle** class -- but forget to update `drawAll()`. What happens? How would you even know something is missing?

The core issue

All shapes share common traits: position (`x, y`), color, `draw()`, `getArea()`, `getPerimeter()`. But Java does not know they are related -- they are independent classes. We need a way to say: "these are all **Shapes**."

The Vision: Before and After

Current: **Object** []

- No common type for shapes
- **instanceof** chain in **drawAll()**
- Adding a shape = editing **Drawing**
- Can accidentally add non-shapes

Goal: **Shape** []

- Common type: **Shape**
- Simple loop: **shapes[i].draw(gc)**
- Adding a shape = new class only
- Only **Shape** objects allowed

To get from left to right, we need two new concepts: **inheritance** (this section) and **abstract classes** (next section).

Two Ways to Relate Classes

We already know **composition** (has-a):

- A **Drawing** **has** shapes (field of type **Circle**[])
- A **Player** **has** a **Weapon** (field of type **Weapon**)

Now we learn **inheritance** (is-a):

- A **Circle** **is a** Shape
- A **Rectangle** **is a** Shape
- A **Line** **is a** Shape

Key difference

Has-a = "I contain another object" (composition)

Is-a = "I am a specialized version of another class" (inheritance)

The `extends` Keyword

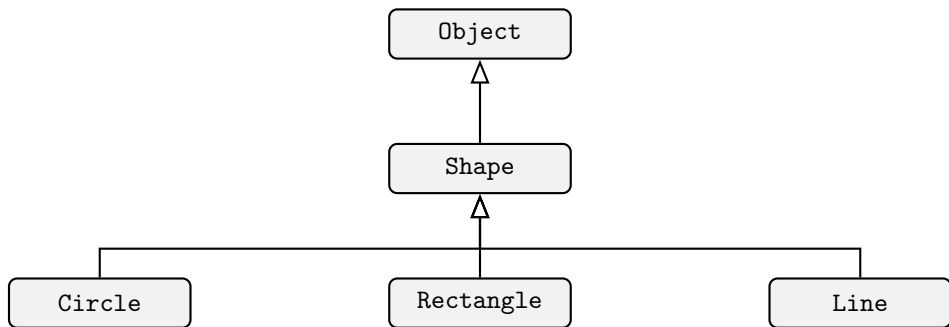
In Java, `extends` creates an inheritance relationship:

```
1  class Shape {  
2      private double x;  
3      private double y;  
4  
5      Shape(double x, double y) {  
6          this.x = x;  
7          this.y = y;  
8      }  
9  }
```

```
1  class Circle extends Shape {  
2      double radius;  
3  
4      Circle(double x, double y,  
5          double radius) {  
6          super(x, y);  
7          this.radius = radius;  
8      }  
9  }
```

`Circle extends Shape`: Circle **is a** Shape -- inherits everything from `Shape` and can add more.

The Inheritance Hierarchy



Every **Shape** **is an** **Object**.

Every **Circle**, **Rectangle**, and **Line** **is a** **Shape** -- and therefore also an **Object**.

What Gets Inherited?

When **Circle** extends **Shape**, Circle gets:

Member	Inherited?	Details
Fields (x , y)	Yes	Circle has x and y
Methods (getX() , ...)	Yes	Circle can call them
Constructors	No	Must write own; calls super()
private members	Exist, but not accessible	Use getters/setters

Example -- private members: **Circle** has an **x** field from **Shape**, but cannot write **this.x** inside Circle's code. It must call **this.getX()** instead. The field exists; the direct name is off-limits.

Think of it as

The parent class provides a **foundation**. The child class adds **specialization** on top.

The **super** Keyword

super = parent class (like **this** = current object).

```
1  class Shape {
2      private double x, y;
3      Shape(double x, double y) {
4          this.x = x; this.y = y;
5      }
6  }
7
8  class Circle extends Shape {
9      private double radius;
10     Circle(double x, double y, double r) {
11         super(x, y); // parent ctor
12         this.radius = r;
13     }
14     String info() {
15         return super.toString() + " r=" + radius;
16     }
17 }
```

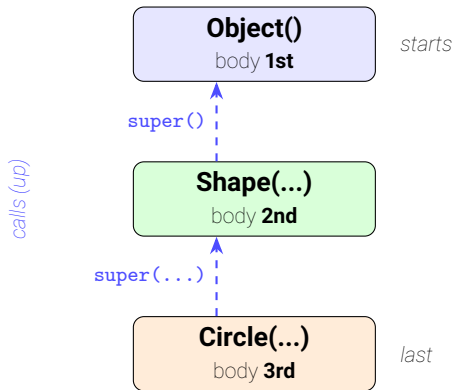
-- **super(...)** in constructor: call parent's constructor (must be first line).

-- Parent only has parameterized constructor $\Rightarrow$ omitting **super(...)** is a compile error.

-- **super.methodName()**: call parent's method.

Constructor Chaining

Child calls **super()** **upward**; parent body finishes before child's runs.



- **super()** goes **up**: child invokes parent before its own work.
- Bodies complete **top-down**: parent fully ready before child continues.

Quick Check: Predict the Output

```
1  class Shape {  
2      Shape(double x, double y) {  
3          IO.println("Shape at (" + x + ", " + y + ")");  
4      }  
5  }  
6  class Circle extends Shape {  
7      Circle(double x, double y, double r) {  
8          super(x, y);  
9          IO.println("Circle, r=" + r);  
10     }  
11 }  
12  
13 Circle c = new Circle(100, 200, 50);
```

What does this print, and in what order?

Quick Check: Predict the Output - Answer

```
1  Shape at (100.0,200.0)
2  Circle, r=50.0
```

Explanation

Circle() calls **super(x, y)** first → **Shape**'s constructor runs and prints the first line. Only then does the **Circle** constructor body continue, printing the second line. The parent is always fully initialized before the child.

Method Overriding with @Override

A child class can **replace** a parent's method with its own version:

```
1  class Shape {
2      // private x, y -- use getters even inside Shape's own methods
3      String describe() { return "shape at (" + getX() + ", " + getY() + ")"; }
4  }
5
6  class Circle extends Shape {
7      @Override
8      String describe() { return "circle with radius " + this.radius; }
9  }
```

- **@Override** tells the compiler: "I am replacing a parent method"
- Same name, same parameters, same return type as the parent method
- The child version **replaces** the parent version for that class

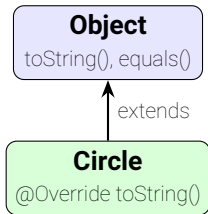
You Already Know This!

You have been overriding methods since Session 1 -- you just didn't know the terminology:

```
-- @Override public String toString() -- overrides Object.toString()  
-- @Override public boolean equals(Object obj) -- overrides Object.equals()
```

The connection

Every class in Java implicitly extends **Object**. When you wrote `@Override toString()`, you were overriding a method inherited from **Object**. Now you see *why* it works -- it is inheritance!



Why It Works Without `@Override`

`@Override` is **optional**. Overriding happens whenever a subclass defines a method with the **same signature** as a parent method.

- **Without `@Override`:** Java still uses the subclass method when you call it on a subclass instance. The behavior is the same.
- **With `@Override`:** The compiler checks that you really are overriding something. If you have a typo (ex. "toString vs. toStringing") the method name or change the parameters, you get a **compile error** instead of accidentally defining a new method.

Best practice

Use `@Override` whenever you intend to override. It protects you from mistakes and makes the intent clear to readers.

Spot the Bug!

```
1  class Shape {  
2      private double x, y;  
3      Shape(double x, double y) { this.x = x; this.y = y; }  
4      double getArea() { return 0; }  
5  }
```

Each Circle snippet below has one bug. Find it.

```
1  class Circle extends Shape {  
2      private double radius;  
3      Circle(double x, double y,  
4          double radius) {  
5          this.radius = radius;  
6      }  
7  }
```

```
1  class Circle extends Shape {  
2      private double radius;  
3      @Override  
4      double getArea(double r) {  
5          return Math.PI * r * r;  
6      }  
7  }
```

Spot the Bug! - Answer

Bug 1: Missing **super()** call

The constructor does not call **super(x, y)** -- the parent (**Shape**) is never initialized. If **Shape** has no default constructor, this is a **compile error**. Fix: add **super(x, y);** as the first line.

Bug 2: Wrong method signature

@Override **getArea(double r)** has a **parameter** that the parent method does not have. This is not overriding -- it is a new method with a different signature. The compiler will report an error because **@Override** does not match any parent method. Fix: remove the parameter and use **this.radius** instead.

Bonus question

Which bug does the compiler always catch? Which could **silently** produce wrong results if **Shape** happened to have a no-arg constructor?

Spot the Bug! -- Bonus Answer

- **Compiler always catches:** Bug 2. `@Override` must match a parent method; `getArea(double r)` does not match `Shape.getArea()`, so you get a compile error.
- **Can be silent:** Bug 1. If `Shape` had a no-arg constructor, the compiler would insert `super();` and the code would compile --- but `x` and `y` would never be set, giving wrong or default values.

Terminology

Term	Meaning
Parent class / Superclass	The class being extended (Shape)
Child class / Subclass	The class that extends (Circle)
extends	Keyword to create inheritance
super	Reference to the parent class
Overriding	Replacing a parent method in the child
Inheriting	Getting fields/methods from the parent

You may hear “base class” (= parent) and “derived class” (= child) in other languages (C++, C#). In Java, the standard terms are superclass and subclass.

Recap - Inheritance

Before we continue, let's make sure the key ideas are clear:

- Our **Drawing** class uses **Object[]** and **instanceof** chains -- this does not scale
- **Inheritance** (is-a): a child class **extends** a parent class
- The child inherits fields and methods; constructors are **not** inherited
- **super(...)** calls the parent constructor (must be first statement)
- **@Override** replaces a parent method with the child's version
- Constructor chaining: parent initialized before child

Any questions before we move to abstract classes?

Can We Create a Generic Shape?

We want a **Shape** parent class. But what would this mean?

```
1  Shape s = new Shape(10, 20, Color.RED);
2  s.getArea();           // area of... what?
3  s.getPerimeter();      // perimeter of... what?
4  s.draw(gc);            // draw... what?
```

- A “generic shape” has no geometry -- it makes no sense to create one
- We **cannot** compute area, perimeter, or draw something with no specific form
- But we **want** to say that every shape *must have* these methods

Can We Create a Generic Shape?

We want a **Shape** parent class. But what would this mean?

```
1  Shape s = new Shape(10, 20, Color.RED);
2  s.getArea();           // area of... what?
3  s.getPerimeter();      // perimeter of... what?
4  s.draw(gc);            // draw... what?
```

- A “generic shape” has no geometry -- it makes no sense to create one
- We **cannot** compute area, perimeter, or draw something with no specific form
- But we **want** to say that every shape *must have* these methods

The solution

Make **Shape** an **abstract class**: you can extend it, but you cannot create an instance of it directly. It is a **blueprint**, not a finished product.

The `abstract` Keyword

```
1  abstract class Shape {
2      private double x, y;
3      private Color color;
4
5      Shape(double x, double y, Color color) {...}
6
7      // concrete methods -- shared by all shapes
8      public double getX() { return this.x; }
9      public double getY() { return this.y; }
10     public Color getColor() { return this.color; }
11
12     // abstract methods -- each shape implements differently
13     public abstract double getArea();
14     public abstract double getPerimeter();
15     public abstract void draw(GraphicsContext gc);
16 }
```

Concrete vs. Abstract Methods

An abstract class can have **both** types of methods:

Concrete methods

- Have a body (implementation)
- Shared by all subclasses
- Example: **getX()**, **getColor()**
- Same logic for every shape

Abstract methods

- No body -- just a signature
- Subclasses **must** implement them
- Example: **getArea()**, **draw()**
- Different logic for each shape

Rule

If a class has **any** abstract method, the class itself **must** be declared **abstract**. You cannot have an abstract method in a non-abstract class.

What You Cannot Do with Abstract Classes

```
1  Shape s = new Shape(10, 20, Color.RED); // COMPILE ERROR!  
2  // Cannot instantiate the abstract class Shape
```

- You **cannot** create an instance of an abstract class with **new**
- You **can** use it as a variable type: **Shape s = new Circle(...);**
(This works because Circle IS-A Shape -- more on this in EH 3)
- You **can** extend it: **class Circle extends Shape {...}**

Abstract Classes: Analogy & Misconception

Analogy

An abstract class is like a **job description**: it lists what must be done (`getArea()`, `draw()`) but does not do the work itself. You need to hire a specific employee (subclass) to actually do it.

Common misconception

“Abstract means the class is broken or incomplete.” -- No: abstract means it is **intentionally not meant to stand alone**. It is a deliberate design decision.

Quick Check: Abstract or Concrete?

Which of these methods in Shape should be abstract, and which concrete?

1. `getArea()` -- returns the shape's area
2. `getColor()` -- returns the shape's color
3. `draw(GraphicsContext gc)` -- draws the shape on screen
4. `getX()` -- returns the x position
5. `getPerimeter()` -- returns the perimeter
6. `toString()` -- returns a text description

Think: does every shape compute this the same way, or differently?

Quick Check: Abstract or Concrete? - Answer

Method	Type	Why?
<code>getArea()</code>	Abstract	Different formula per shape
<code>getColor()</code>	Concrete	Same for all shapes (return field)
<code>draw(gc)</code>	Abstract	Each shape draws differently
<code>getX()</code>	Concrete	Same for all shapes (return field)
<code>getPerimeter()</code>	Abstract	Different formula per shape
<code>toString()</code>	Could be both	Concrete default OK; subclasses can override

General rule: If the implementation depends on the specific shape type → **abstract**. If it is the same for all shapes → concrete.

The Refactoring Plan

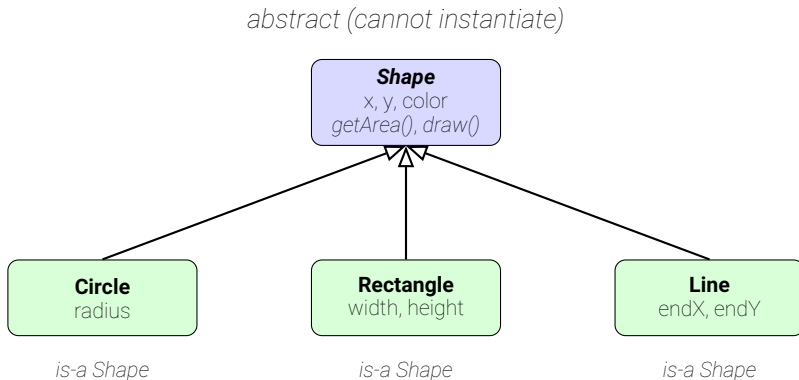
We will transform our independent shape classes into a hierarchy, step by step:

1. Create **abstract class Shape** with common fields and constructor
2. Add concrete methods: `getX()`, `getY()`, `getColor()`, `setColor()`
3. Add abstract methods: `getArea()`, `getPerimeter()`, `draw(GraphicsContext gc)`
4. Make **Circle extends Shape** -- implement abstract methods
5. Make **Rectangle extends Shape** -- same pattern
6. Make **Line extends Shape** -- interesting edge case
7. Rewrite **Drawing** to use **Shape[]** instead of **Object[]**

The goal: eliminate the `instanceof` chain entirely.

The Target Hierarchy

Before we write any code -- here is where we are heading:



Italic = abstract. Hollow triangle = inheritance. We will now implement this step by step.

Step 1--3: The Abstract **Shape** Class

```
1  package at.fhooe.ald.shapes;
2  import javafx.scene.canvas.GraphicsContext;
3  import javafx.scene.paint.Color;
4
5  public abstract class Shape {
6      private double x, y;
7      private Color color;
8
9      public Shape(double x, double y, Color color) {
10         this.x = x;
11         this.y = y;
12         this.color = color;
13     }
14
15     public double getX() { return this.x; }
16     public double getY() { return this.y; }
17     public Color getColor() { return this.color; }
18     public void setColor(Color color) { this.color = color; }
19
20     public abstract double getArea();
21     public abstract double getPerimeter();
22     public abstract void draw(GraphicsContext gc);
23 }
```

Step 4: Circle extends Shape

```
1 public class Circle extends Shape {
2     private double radius;
3
4     public Circle(double x, double y, double radius, Color color) {
5         super(x, y, color); // initialize Shape fields
6         this.radius = radius;
7     }
8
9     public double getRadius() { return this.radius; }
10
11     @Override
12     public double getArea() {
13         return Math.PI * this.radius * this.radius;
14     }
15     @Override
16     public double getPerimeter() {
17         return 2 * Math.PI * this.radius;
18     }
19     @Override
20     public void draw(GraphicsContext gc) {
21         gc.setFill(this.getColor());
22         double d = this.radius * 2;
23         gc.fillOval(this.getX() - this.radius,
24                     this.getY() - this.radius, d, d);
25     }
26 }
```

What Changed in Circle?

Compared to the old **Circle** class:

Before	After
<code>class Circle</code>	<code>class Circle extends Shape</code>
Fields: <code>x</code> , <code>y</code> , <code>color</code> , <code>radius</code>	Fields: only <code>radius</code> (rest inherited)
Own <code>getX()</code> , <code>getY()</code> , <code>getColor()</code>	Inherited from <code>Shape</code>
Constructor sets all fields	Constructor calls <code>super(x, y, color)</code>
<code>getArea()</code> , <code>draw()</code> defined freely	<code>@Override</code> -- must match abstract signature

Less code, more structure. Circle only defines what is *unique* to circles.

What This Means for Code Quality

The Shape hierarchy (inheritance + abstract methods) improves real code in three ways:

- **Less duplication** -- Common data (`x`, `y`, `color`) and behaviour (getters, etc.) live once in `Shape`. Each subclass only adds what is specific (e.g. `radius`, `getArea()`).
- **Better maintainability** -- Change or add something in `Shape` (e.g. `getCenter()`, how `color` is stored) and all subclasses get it. One place to fix or extend.
- **Clearer quality** -- Abstract methods define a **contract**: every shape must implement `getArea()`, `draw()`, etc. No “forgot to implement”; the compiler enforces it. Same API for all shapes.

Your Turn: Refactor Line

Think about this with a neighbour

If you had to write **Line** **extends** **Shape** from scratch:

1. What fields does **Line** need beyond what **Shape** provides?
2. What does **super(...)** in Line's constructor look like?
3. What does **getArea()** return for a line?
4. How does **draw()** differ from Circle and Rectangle?

Hint

Shape already stores the **start** point (**x**, **y**). What additional coordinates does a Line need? What does "area" mean for a one-dimensional object?

Discuss with your neighbour, then we look at the solution together.

Step 6: Line extends Shape -- Solution

```
1  public class Line extends Shape {
2      private double endX, endY;
3
4      public Line(double startX, double startY, double endX,
5                  double endY, Color color) {
6          super(startX, startY, color);
7          this.endX = endX;
8          this.endY = endY;
9      }
10
11     @Override
12     public double getArea() {
13         return 0; // a line has no area!
14     }
15     // other abstract methods (not shown for brevity)
16 }
```


Discussion: Is a Line Really a Shape?

Line extends **Shape** raises an interesting question:

- **getArea()** returns 0 -- a line has no area
- **getPerimeter()** returns the length -- is that really a “perimeter”?
- A line uses **strokeLine()** while circles and rectangles use **fill...()**

Is this a problem?

It is a **design trade-off**. The benefit: **Line** can be stored in **Shape[]** and drawn in the same loop. The cost: **getArea()** returns a meaningless value. For our **Drawing** application, the benefit outweighs the cost. In a geometry library, you might decide differently.

This is a real-world design discussion -- there is no perfect answer. The key is to think about it consciously.

Step 7: The New **Drawing** Class

Now the payoff -- compare the old and new **drawAll()**:

Before (**Object[]**)

```
1  public void drawAll(GraphicsContext gc) {
2      for (int i = 0; i < count; i++) {
3          if (shapes[i] instanceof Circle c) {
4              c.draw(gc);
5          } else if (shapes[i]
6                  instanceof Rectangle r) {
7              r.draw(gc);
8          } else if (shapes[i]
9                  instanceof Line l) {
10             l.draw(gc);
11         }
12     }
13 }
```

After (**Shape[]**)

```
1  public void drawAll(GraphicsContext gc) {
2      for (int i = 0; i < count; i++) {
3          this.shapes[i].draw(gc);
4      }
5  }
```

No **instanceof**. No type checks. Each shape knows how to draw itself.

Using the New Drawing

```
1  Drawing drawing = new Drawing(10);
2  drawing.add(new Circle(200, 200, 50, Color.RED));
3  drawing.add(new Rectangle(400, 100, 120, 80, Color.BLUE));
4  drawing.add(new Line(100, 400, 500, 350, Color.GREEN));
5
6  drawing.drawAll(gc); // each shape draws itself!
7  IO.println("Total area: " + drawing.getTotalArea());
```

What we gained

- **add()** only accepts **Shape** objects -- type safety
- **drawAll()** is 3 lines -- no **instanceof**
- **getTotalArea()** works for any mix of shapes
- Adding a new shape type requires **zero changes** to **Drawing**

Recap - Abstract Classes

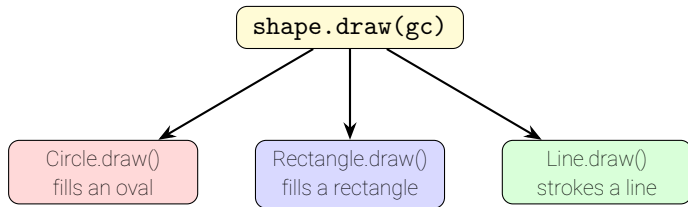
- An **abstract** class cannot be instantiated with **new**
- Abstract methods have no body -- subclasses **must** implement them
- An abstract class can have both abstract **and** concrete methods
- We refactored: **Shape** (abstract) ← **Circle, Rectangle, Line**
- **Drawing** now uses **Shape []** -- no more **instanceof** chains
- Adding new shapes requires zero changes to existing code

Any questions before we move to polymorphism?

What is Polymorphism?

Polymorphism = “one method call, many behaviors” (from Greek: *poly* = many, *morph* = form)

When you call `shape.draw(gc)`, the **actual behavior** depends on what kind of shape it is:



Same method call, different behavior -- determined at **runtime**.

Dynamic Dispatch

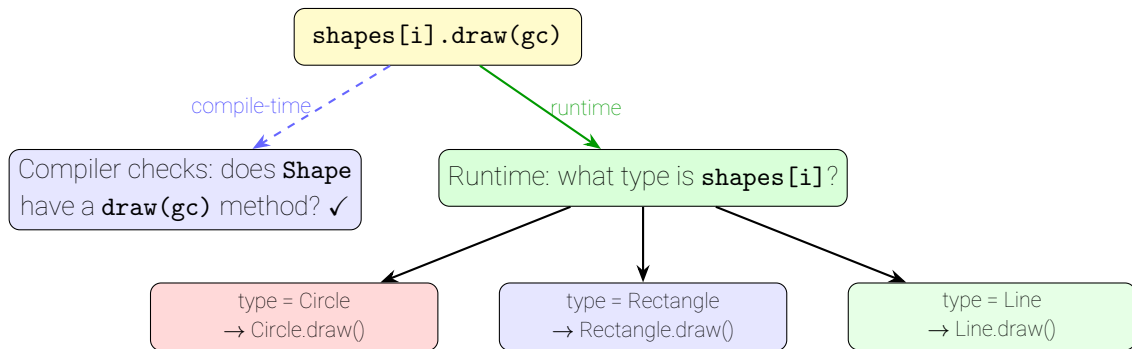
Java decides which `draw()` to call based on the **actual object type**, not the variable type:

```
1  Shape s1 = new Circle(100, 100, 50, Color.RED);
2  Shape s2 = new Rectangle(200, 200, 80, 60, Color.BLUE);
3
4  s1.draw(gc); // calls Circle's draw() -- draws an oval
5  s2.draw(gc); // calls Rectangle's draw() -- draws a rectangle
```

- **Variable type:** `Shape` -- what the compiler sees
- **Object type:** `Circle` or `Rectangle` -- what is actually in memory
- At runtime, Java looks at the **object type** to decide which method to run
- This is called **dynamic dispatch** (or “late binding”)

The `Drawing` class only knows about `Shape`. It calls `shapes[i].draw(gc)`. Java automatically picks the right version for each shape.

Dynamic Dispatch: How it Works at Runtime



Blue (compile-time): compiler only checks the **variable type** (**Shape**) -- does the method exist?

Green (runtime): Java looks at the **actual object** to pick the right implementation.

Quick Check: Predict the Output

```
1  Shape[] shapes = new Shape[3];
2  shapes[0] = new Circle(0, 0, 5, Color.RED);
3  shapes[1] = new Rectangle(0, 0, 4, 3, Color.BLUE);
4  shapes[2] = new Circle(0, 0, 10, Color.GREEN);
5
6  for (int i = 0; i < shapes.length; i++) {
7      IO.println(shapes[i].getArea());
8  }
```

What does this print? Which `getArea()` runs for each element?

Quick Check: Predict the Output - Answer

```
1    78.539...    // Circle: PI * 5^2
2    12.0         // Rectangle: 4 * 3
3    314.159...   // Circle: PI * 10^2
```

Explanation

Each element in the array is a **Shape** variable, but the actual objects are **Circle** and **Rectangle**. Java uses **dynamic dispatch**: at runtime, it calls the correct **getArea()** based on the actual type. The loop does not need to know which shape it is dealing with.

Polymorphism in the Drawing Loop

This is the power of polymorphism -- our `drawAll()` works for **any** shape:

```
1  public void drawAll(GraphicsContext gc) {
2      for (int i = 0; i < this.count; i++) {
3          this.shapes[i].draw(gc);
4          // Circle? -> fillOval
5          // Rectangle? -> fillRect
6          // Line? -> strokeLine
7          // Triangle? -> fillPolygon (if we add it later!)
8      }
9  }
```

- **Drawing** does not know (or care) which specific shape types exist
- Each shape handles its own drawing logic
- This is **polymorphism**: one method call (`draw`), many forms (different shapes)

Variable Type vs Object Type

```
1    Shape s = new Circle(100, 100, 50, Color.RED);
```

Variable type: **Shape**

- What the **compiler** checks
- Limits which methods you can **call**
- **s.getArea()** -- OK
- **s.getRadius()** -- **compile error**

Object type: **Circle**

- What is actually **in memory**
- Determines which version **runs**
- **s.getArea()** → **Circle.getArea()**
- Full Circle (incl. **radius**) is in memory

Variable Type vs Object Type -- Misconception

Common misconception

"If **s** is of type **Shape**, it only contains Shape data." -- **Wrong**. The full **Circle** object is in memory; the variable type just gives a narrower *view*. If you need **getRadius()**: declare as **Circle c = new Circle(...)**; or cast **((Circle) s).getRadius()** -- but casting is usually a design smell.

Rule: Variable type = what you can call. Object type = what runs.

Adding a New Shape: Triangle

To add a **Triangle**, we only need to:

1. Create a new class that **extends Shape**
2. Implement the three abstract methods
3. Done -- **Drawing** works without any changes

```
1  public class Triangle extends Shape {
2      private double base, height;
3
4      public Triangle(double x, double y, double base, double height, Color color) {...}
5      @Override
6      public double getArea() {
7          return 0.5 * this.base * this.height;
8      }
9      @Override
10     public double getPerimeter() { /* ... */ return 0; }
11     @Override
12     public void draw(GraphicsContext gc) { /* ... */ }
13 }
```

Triangle Works Everywhere -- Immediately

```
1  Drawing drawing = new Drawing(10);
2  drawing.add(new Circle(100, 100, 50, Color.RED));
3  drawing.add(new Rectangle(300, 100, 80, 60, Color.BLUE));
4  drawing.add(new Triangle(500, 100, 70, 90, Color.GREEN));
5
6  drawing.drawAll(gc); // Triangle draws itself too!
7  IO.println("Total area: " + drawing.getTotalArea());
```

Zero changes to existing code

Drawing.drawAll(), **Drawing.add()**, **Drawing.getTotalArea()** -- none of these needed editing.

The Open-Closed Principle

Design Principle **Open-Closed Principle** (OCP)

Open for extension -- you can add new shapes without changing existing code.

Closed for modification -- **Drawing** and all existing shapes are untouched.

When you add Triangle...	What changes?
Triangle.java	New file only
Drawing.java	Nothing
Circle.java, Rectangle.java	Nothing
Your <code>main()</code> / <code>ShapeApp</code>	Only the <code>add()</code> call

Why this matters: the more existing code you *avoid* editing, the fewer places you can introduce bugs. New features should not break existing features.

Inheritance Rules Summary

Rule	Details
Single inheritance	A class can extend only one class
Constructor chaining	Child must call super(...) first
All classes extend Object	If no extends , Java adds extends Object
abstract class	Cannot be instantiated; can have abstract methods
abstract method	No body; subclass must implement
@Override	Replace a parent method; compiler checks signature
Polymorphism	Variable type = what you can call; object type = what runs

Single inheritance limitation

A class can only extend **one** other class. What if we want a shape that is both **Drawable** and **Selectable**? Session 4 introduces **interfaces** -- Java's solution to this limitation.

Why Not Multiple Inheritance?

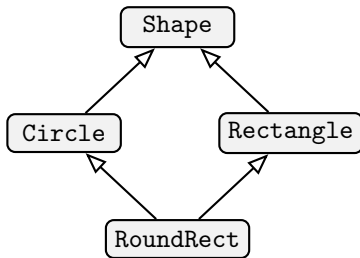
Multiple inheritance = a class extends **more than one** class (e.g. `class Circle extends Shape, Selectable`). Some languages allow it: C++, Python, etc.

Why it is a bad idea:

- **Diamond problem**: e.g. `RoundRect` extends `Circle` and `Rectangle`; both extend `Shape`. Which `printName()` runs? (See next slide.)
- **Ambiguity**: Same method or field in two parents -- which one do you get? Languages add complex rules (e.g. C++ virtual inheritance) that are easy to get wrong.
- **Fragile and hard to maintain**: Changing one parent can break subclasses in non-obvious ways. Debugging “which implementation did I get?” is painful.

Java chose **single inheritance** to keep the model simple and predictable. For “multiple types” we use **interfaces** (next session).

The Diamond Problem



Each class has a method `printName()` that prints its class name. **Shape** prints "Shape", **Circle** overrides to print "Circle", **Rectangle** to print "Rectangle". **RoundRect** extends both **Circle** and **Rectangle**. You have `RoundRect r = new RoundRect(); r.printName();` --- **does it print "Circle" or "Rectangle"?** There is no obvious right answer. Languages that allow multiple inheritance need rules to decide.

How Python Solves the Diamond: MRO

Python allows multiple inheritance. It fixes the diamond by defining a **Method Resolution Order (MRO)**: a single, fixed order in which to look for methods.

- When you call `r.printName()` on a `RoundRect`, Python does not “choose” between `Circle` and `Rectangle` arbitrarily. It walks a **linear** order (e.g. `RoundRect` → `Circle` → `Rectangle` → `Shape`) and uses the **first** class in that order that defines `printName()` --- so you get “Circle” or “Rectangle” depending on the order.
- The order is determined by the **declaration**: `class RoundRect(Circle, Rectangle)` vs `class RoundRect(Rectangle, Circle)` can change which `printName()` runs.
- The algorithm (C3 linearization) keeps the order consistent, but you have to *know* the MRO to understand your code. In Java, with single inheritance, there is only one path --- no such rules needed.

Preview: Next time -- Interfaces

We solved the **Drawing** problem with inheritance. But what if we want:

- Some shapes are **Selectable** (can be clicked)
- Some shapes are **Resizable** (can be dragged to resize)
- Some shapes are **Serializable** (can be saved to a file)
- A **Circle** might be all three!

With single inheritance, **Circle** can only extend **one** class. It already extends **Shape**.

Next session

Java has **interfaces** -- a way to give a class multiple "types" without multiple inheritance. We will also look at JavaFX events (mouse clicks, keyboard input) which use interfaces heavily.

Recap - Polymorphism

- **Polymorphism**: one method call (**draw**), many behaviors -- determined at runtime
- **Dynamic dispatch**: Java looks at the **object type** (not the variable type) to pick the method
- **Variable type** (e.g. **Shape**) = what the compiler lets you *call*
- **Object type** (e.g. **Circle**) = what *actually runs*
- **Open-Closed Principle**: extend without modifying -- add **Triangle**, touch nothing else

Any questions before we formalize our design with UML?

What is UML?

UML (Unified Modeling Language) is a visual language for designing and documenting software:

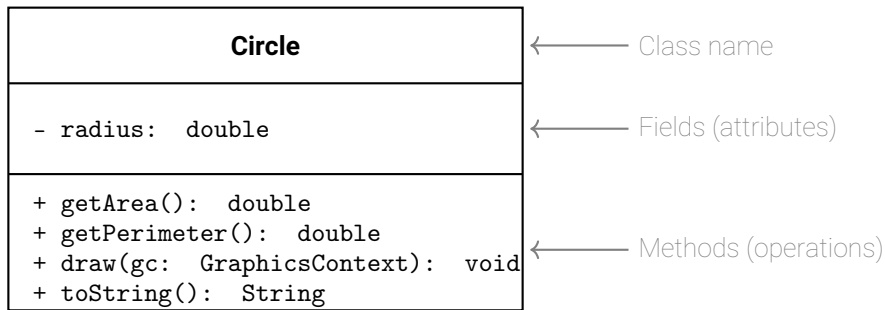
- Standard way to draw class structures, relationships, and interactions
- We focus on **class diagrams** -- the most commonly used UML diagram type
- Useful for planning code before writing it, and for communicating design to others

We keep it minimal

UML has many diagram types and notation details. We learn only what we need: class boxes, fields, methods, and two relationship types (inheritance and composition).

The Class Box

A UML class is drawn as a box with three sections:



*UML rule: each class box shows only its **own** fields/methods. Inherited ones appear only in the parent's box -- not repeated in the child.*

Visibility Notation

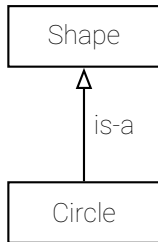
UML uses symbols to show access modifiers:

Symbol	Java keyword	Meaning
+	<code>public</code>	Visible to everyone
-	<code>private</code>	Only the class itself
#	<code>protected</code>	Same package + subclasses
~	(package-private)	Same package only

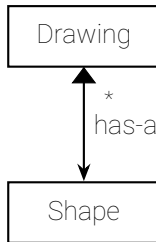
- Fields are usually - (private); methods are usually + (public)
- Abstract classes and methods are written in *italics*
- # (protected): accessible in the same package **and** in subclasses. Think of it as: private = your bedroom, **protected = your house**, public = the street. We don't use it in our shapes, but you will see it in Java libraries.

Relationships: Inheritance and Composition

Inheritance

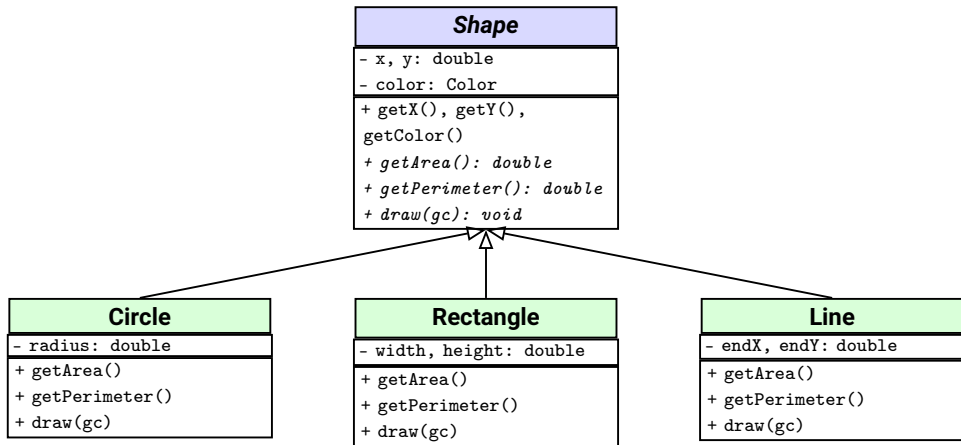


Composition



- **Hollow triangle arrow** = inheritance (is-a): child → parent
- **Filled diamond + arrow** = composition (has-a): whole → part
- The * means "many" (Drawing has many Shapes)

Our Shape Hierarchy as UML



Italic = abstract. Subclasses show their **own** fields and their **implementations** of the abstract methods; inherited concrete methods (`getX`, `getY`, `getColor`) appear only in *Shape*.

UML Notation Summary

Notation	Meaning
Three-section box	Class: name fields methods
<i>Italic name</i>	Abstract class or method
+ / - / #	public / private / protected
Hollow triangle arrow	Inheritance (is-a)
Filled diamond arrow	Composition (has-a)
* / 1 / 0..1	Multiplicity (many / one / zero or one)

Quick Retrieval: Test Yourself

Before looking at the summary -- can you answer these from memory?

1. What keyword creates an inheritance relationship?
2. What is the difference between **abstract** and concrete methods?
3. Why can't you write **new Shape(...)**?
4. What does **super(x, y, color)** do in a constructor?
5. If you add a new shape type, what changes in **Drawing**?

Take some time to think, then we go through the answers.

Summary

- **Inheritance** (is-a): `Circle extends Shape` -- child inherits fields and methods
- `super(...)` calls the parent constructor; `super.method()` calls the parent's method
- **Constructor chaining**: parent initialized before child
- `@Override` replaces a parent method -- compiler checks the signature
- **Abstract classes**: cannot be instantiated; define a contract for subclasses
- **Abstract methods**: no body -- subclasses must implement them
- **Polymorphism**: one method call, different behavior depending on the actual object type
- **UML class diagrams**: class boxes with fields/methods, inheritance arrows, composition arrows
- **Open-Closed Principle**: open for extension (add classes), closed for modification (don't edit `Drawing`)